

TrustChain-FL: A Blockchain-Assisted Reputation Scoring Framework for Robust Federated Learning Against Model Poisoning Attacks

Deepanshu Chauhan¹ , Amitosh Tripathi¹ , Tracy^{1,*} , Saksham Saxena¹ 

¹Department of Computer Science and Engineering, Bennett University, Greater Noida, Uttar Pradesh, India

*Corresponding author: e23cseu1617@bennett.edu.in

Abstract

Federated learning allows geographically distributed participants to collaboratively train a shared machine learning model without transferring raw data to a central server, offering genuine privacy benefits for domains such as healthcare, finance, and edge computing. However, the openness of the gradient exchange process makes federated pipelines vulnerable to poisoning attacks in which compromised clients submit manipulated updates that degrade or subvert global model quality. Existing Byzantine-robust aggregation techniques either require a trusted clean reference dataset at the server, demand knowledge of the poisoning fraction in advance, or provide no auditable record of participant behavior across training rounds. This paper presents TrustChain-FL, a system that addresses these limitations simultaneously by embedding a permissioned blockchain layer into the federated training loop as a tamper-proof reputation store. Each participating client accumulates a dynamic reputation score derived from the cosine similarity between its submitted gradient and a majority-consensus reference signal computed from historically reputable peers. Scores are updated through a closed-form exponential moving average and committed on-chain after every communication round without storing full gradient tensors, keeping the storage footprint at $O(n)$ per round independent of model size. A statistical anomaly detection gate based on Kullback-Leibler divergence quarantines gradient updates whose empirical distribution deviates significantly from the honest-majority pattern before they reach the aggregation step. The aggregator subsequently forms the global update using reputation-weighted averaging, reducing the influence of historically deviant clients without requiring manual exclusion thresholds. Experiments on a ten-client federated simulation using the MNIST and CIFAR-10 benchmarks under label-flipping, scaled gradient, and mixed attack scenarios demonstrate that TrustChain-FL retains test accuracy within two percentage points of a fully trusted no-attack baseline even when thirty percent of clients are adversarial, compared to a degradation of up to nineteen percentage points under standard FedAvg. The blockchain overhead averages 378 milliseconds per round on a local Ganache Ethereum testnet, confirming practical feasibility at the prototype scale while preserving a complete auditable record of all participant contributions.

Keywords: federated learning; blockchain; reputation scoring; model poisoning; anomaly detection; Byzantine robustness; smart contracts; gradient security.

1 Introduction

The rapid proliferation of connected devices, institutional data silos, and privacy-sensitive applications has driven demand for machine learning approaches that do not require centralizing raw data. Federated learning (FL), first formalized by McMahan et al. [1], responded to this demand by distributing the gradi-

ent computation step across participants and aggregating only model updates at a central server. Under this design, each client computes gradients on its local dataset and transmits the resulting weight deltas, never exposing the underlying records. The privacy benefit is genuine and has enabled FL deployments in medical imaging, keyboard prediction, autonomous driving telemetry, and financial fraud detection [2, ?].

Despite its appeal, the federated architecture introduces attack surfaces that do not exist in centralized training. Because the aggregation server receives updates from clients it cannot fully monitor, any participant can independently modify its submitted gradient without detection. This observation motivates a category of threats collectively referred to as model poisoning, where compromised clients deliberately corrupt the training process. Label-flipping attacks systematically mislabel local training data so that the resulting gradient pushes the global model toward incorrect predictions on targeted input classes [3]. Scaled gradient attacks multiply the malicious update by a large factor to dominate the aggregation average and override the contributions of honest participants [4]. Backdoor attacks implant persistent trigger-based misclassification that survives numerous subsequent honest rounds [4]. Free-rider attacks submit zero or noise gradients to collect model improvements without contributing computation. These threats are not merely theoretical: controlled experiments have demonstrated that a single malicious client among ten can cause FedAvg to converge to a poisoned model that misclassifies an entire target class with near-certainty [5].

The research community has proposed several Byzantine-robust aggregation rules as defenses. Krum selects the single update geometrically closest to its neighbors, offering formal fault tolerance bounds [6]. Coordinate-wise median and trimmed-mean aggregators replace the arithmetic average with order statistics that are resistant to outliers [7]. FLTrust introduces trust score weighting based on cosine similarity with a server-computed reference gradient, achieving strong empirical performance on a range of attacks [8]. However, each approach carries constraints that restrict practical applicability. Krum discards all but one update per round, wasting honest contributions. Trimmed-mean requires the poisoning fraction to be known or estimated in advance. FLTrust requires a small clean root dataset at the server, a data dependency that violates the privacy assumptions of many federated deployments and is simply unavailable in cross-silo settings where the server is a neutral coordinator. None of these methods produce a persistent, verifiable record of participant behavior, so a client that alternates between honest and malicious rounds cannot be distinguished from one that is consistently honest.

Blockchain technology provides precisely the audit trail that FL defenses lack. Its append-only,

consensus-validated ledger records every transaction permanently and without the possibility of retroactive modification [?]. Its smart contract layer enables programmable trust logic that executes deterministically and can be independently verified by any network participant [?]. Prior proposals to integrate blockchain with FL have struggled to find a practical operating point. Systems that store full gradient tensors on-chain create storage overheads proportional to model size multiplied by client count multiplied by round count, which quickly becomes infeasible for any non-trivial model. Systems that use zero-knowledge proofs for gradient correctness verification require seconds of computation per client per round, making them impractical for training pipelines that must complete hundreds of rounds [9]. Systems that store only gradient hashes for post-hoc audit purposes but delegate verification to an off-chain trusted node reintroduce the very privileged party that blockchain was intended to eliminate [10].

TrustChain-FL is designed to occupy the practical middle ground between these extremes. It records only a compact reputation tuple and a SHA-256 gradient hash per client per round, keeping on-chain storage at $O(n)$ regardless of model size. Reputation scores are computed using cosine similarity against a reference gradient derived entirely from the current round’s honest-majority consensus, removing the server data dependency. A KL-divergence anomaly gate provides a first-pass defense before reputation scoring, quarantining clearly anomalous updates immediately. The aggregation step then applies reputation-weighted averaging, which degrades to standard FedAvg when all clients are honest, ensuring no accuracy cost in the benign setting. The complete history of reputation scores, quarantine decisions, and gradient hashes is publicly readable from the blockchain, enabling any participant to verify that aggregation was performed correctly without trusting the aggregator’s claims.

The main contributions of this work are the following. First, a dynamic on-chain reputation scoring mechanism that requires no server-side clean dataset and is updated through a closed-form exponential moving average computable in constant time per round. Second, a KL-divergence anomaly detection gate that operates before aggregation and records quarantine decisions on-chain for audit purposes. Third, a reputation-weighted aggregation rule that reduces to standard FedAvg under honest conditions and introduces no accuracy tax in the be-

nign setting. Fourth, an end-to-end prototype built with PyTorch, Web3.py, Ganache, and Flower, with empirical measurements of model accuracy, detection rate, false positive rate, and blockchain latency overhead under three adversarial configurations across two benchmark datasets. Fifth, a scalability analysis examining how blockchain overhead grows with client count and identifying transaction batching as a practical path toward larger deployments.

The remainder of this paper is organized as follows. Section II reviews related work across three bodies of literature: adversarial FL, Byzantine-robust aggregation, and blockchain integration. Section III defines the threat model and system assumptions. Section IV describes the TrustChain-FL architecture. Section V presents the scoring, detection, and aggregation algorithms with formal definitions. Section VI details the implementation environment, dataset configuration, and attack simulation setup. Section VII reports and discusses experimental results. Section VIII analyzes limitations and future extensions, and Section IX concludes the paper.

2 Related Work

2.1 Foundations of Federated Learning

McMahan et al. introduced the FedAvg algorithm as the canonical FL baseline [1]. FedAvg distributes one or more epochs of stochastic gradient descent to each client, collects the resulting weight deltas, and forms the new global model as a dataset-size-weighted average. The algorithm was shown to converge reliably on image classification and language modeling tasks under independently and identically distributed data partitions, and approximately under mild heterogeneity. Subsequent work by Li et al. established convergence guarantees for non-IID data under bounded gradient dissimilarity conditions [11], confirming that FedAvg is theoretically sound even when clients hold statistically different local datasets. The same survey [2] categorized the open challenges in FL into data heterogeneity, systems heterogeneity, and privacy and security, identifying security against malicious participants as one of the least-solved problems in the field. More recent work has extended FL to cross-silo settings involving institutions rather than devices [?], where the stakes of a poisoning attack are substantially higher because the model may be used in safety-critical applications such as radiology diagnosis or financial risk scoring.

2.2 Adversarial Attacks on Federated Learning

Bhagoji et al. were among the first to analyze FL systematically from an adversarial perspective [5]. They demonstrated that a single malicious client can conduct a model replacement attack by scaling its local model update to replace the global model with the attacker’s desired target, bypassing FedAvg’s averaging because the scale factor amplifies the malicious contribution above the sum of all honest ones. Bagdasaryan et al. extended this line of work to backdoor attacks, showing that a compromised client can embed a pixel-pattern trigger into the global model that causes misclassification of any input carrying the trigger while preserving normal accuracy on clean inputs [4]. Critically, they demonstrated that such backdoors survive many subsequent rounds of honest FedAvg because the constrained optimization used to craft the malicious update preserves model performance on the main task, making detection by loss-based server-side inspection nearly impossible.

Fung et al. introduced FoolsGold, which detects Sybil-style attacks by reducing the contribution weight of clients submitting highly similar gradient updates [3]. While effective against coordinated Sybil adversaries, FoolsGold does not address the case of a single powerful attacker using scaled gradients, where the malicious update is explicitly designed to be dissimilar from honest updates rather than similar to them. Wang et al. investigated edge-case malicious gradients that avoid geometric outlier detection by staying within the convex hull of honest updates, showing that purely distance-based defenses can be evaded by a sufficiently adaptive adversary [?]. This motivates the use of historical behavioral records as an additional signal, which is the core insight behind TrustChain-FL’s reputation layer.

2.3 Byzantine-Robust Aggregation Rules

The machine learning theory community formalized Byzantine fault tolerance for distributed gradient descent in a series of influential papers. Blanchard et al. proposed Krum [6], a selection rule that identifies the update minimizing the sum of squared distances to its $n - f - 2$ nearest neighbors, where f is the assumed number of Byzantine clients. Krum provides a formal guarantee that the selected update lies within a bounded distance of the true gradient direction when $f < n/2$. However, selecting only one

update per round introduces substantial variance and wastes the signal from the remaining honest clients, which limits convergence speed in practice. Multi-Krum [6] selects a fixed number of updates rather than one, improving sample efficiency at the cost of a weaker theoretical guarantee.

Yin et al. proposed coordinate-wise median and trimmed-mean aggregators [7]. The median aggregator computes the median of each model parameter across clients independently, providing a breakdown point of up to $n/2$ Byzantine clients at the cost of discarding the full information content of honest updates. The trimmed-mean aggregator removes the β fraction of extreme values at each coordinate before averaging, where β must be specified in advance. Both rules assume that the adversary does not know the honest clients' updates when designing its attack, an assumption that is increasingly questioned in the literature on adaptive adversaries.

Cao et al. proposed FLTrust [8], which addresses the adaptive adversary concern by anchoring trust scores to a server-computed gradient rather than to the geometric relationships among client updates. The server computes its own gradient on a small root dataset and assigns each client a trust score equal to the ReLU-clipped cosine similarity between the client's update and the server gradient. The final aggregation applies trust-score-weighted averaging with normalized magnitudes. FLTrust achieves state-of-the-art robustness across a wide range of attack types in controlled experiments. Its principal limitation is the root dataset requirement: in settings where the server is a neutral coordinator without labeled data, or where the root dataset could itself be targeted by a data poisoning attack, the defense is inapplicable. TrustChain-FL replaces the root dataset with the on-chain consensus of historically reputable clients, eliminating this dependency.

2.4 Blockchain in Distributed Machine Learning

The use of blockchain as infrastructure for distributed machine learning was proposed independently by several groups around 2019–2020. Kim et al. [12] proposed replacing the central FL server with a blockchain network in which gradient exchanges and aggregation occur entirely through smart contract calls. Their design provides decentralization and prevents a single coordinator from manipulating results, but does not address the underlying problem of mali-

cious gradient submissions, since the on-chain aggregation still applies uniform averaging to all received updates. Majeed and Hwang [?] proposed FLchain, which uses blockchain for audit logging of FL transactions but delegates both aggregation and verification to off-chain nodes, making the blockchain a passive record-keeper rather than an active participant in trust management.

Xu et al. introduced VerifyNet [9], which uses homomorphic hash functions to allow the server to verify gradient correctness without viewing plaintext updates, combined with blockchain logging of verification results. The cryptographic machinery provides stronger guarantees than cosine similarity scoring but requires several seconds of computation per client per round, which is impractical for training pipelines requiring hundreds of rounds. Weng et al. proposed DeepChain [10], which stores gradient hashes on-chain for post-hoc audit and uses blockchain-based incentive tokens to reward contributing clients. The actual correctness of gradients is delegated to an off-chain verifier, reintroducing a trusted third party. Kang et al. [?] proposed a multi-weight subjective logic model for reputation management in FL, combining data quality, model contribution, and historical credibility scores. Their system operates off-chain with a trusted manager node; TrustChain-FL achieves comparable reputation modeling through an on-chain smart contract, removing the manager dependency.

3 Threat Model

3.1 System Participants

TrustChain-FL involves three logical roles. *Clients* hold private local datasets and perform local gradient computation. Each client is identified by a unique Ethereum address registered with the aggregator before the first round. The *aggregator* receives gradient updates, applies the anomaly gate and reputation-weighted aggregation, distributes the updated global model, and writes results to the blockchain. The *blockchain layer* consists of one or more Ethereum-compatible nodes executing the `ReputationLedger` smart contract; in the prototype, a single Ganache node serves this role. In a production deployment, the blockchain nodes would be geographically distributed and operated by mutually distrusting parties, making the ledger genuinely decentralized.

3.2 Adversary Model

An adversary controls a subset $\mathcal{M} \subset \mathcal{C}$ of clients, where $|\mathcal{M}|/|\mathcal{C}| = \beta$ is the poisoning fraction. We assume $\beta < 0.5$, consistent with the standard Byzantine fault tolerance literature. The adversary has complete knowledge of its own local data and the current global model parameters received from the aggregator, but cannot observe other clients' local datasets or submitted gradient tensors before they are received by the aggregator. The aggregator and blockchain infrastructure are honest-but-curious: they execute the protocol faithfully and do not collude with the adversary, but they may log all information visible to them.

3.3 Attack Taxonomy

Three attack types are evaluated in this work.

Label-flipping (LF): Malicious clients relabel a fixed fraction of their local training examples before computing gradients. In the experiments, all instances of class 1 are relabeled as class 7. This causes the malicious gradient to point in a direction that increases loss on the clean global objective while decreasing loss on the corrupted local objective, gradually biasing the global model toward misclassifying the targeted class.

Scaled gradient (SG): Malicious clients compute their gradients honestly but multiply the resulting tensor by a scalar factor $\gamma \gg 1$ before submission. With $\gamma = 10$, a single malicious client among ten contributes the equivalent of ten honest clients in the aggregation average, dominating the update direction unless a robust aggregation rule reduces its weight.

Mixed (MIX): A combination of label-flipping and scaled gradient attacks is deployed across different malicious clients simultaneously. This scenario tests whether the anomaly gate and reputation scoring remain effective when different types of anomalous updates appear in the same round.

Sybil attacks, network-level tampering, and server compromise are outside the scope of this work and are identified as directions for future extension.

4 System Architecture

4.1 End-to-End Pipeline

Figure 1 illustrates the TrustChain-FL pipeline. The system operates in discrete communication rounds indexed by t . At the start of round t , the aggregator

broadcasts the current global model $\mathbf{w}^{(t)}$ to all registered clients. Each client i trains locally for E epochs and transmits its gradient update $\Delta \mathbf{w}_i^{(t)}$ together with a SHA-256 hash $h_i^{(t)} = \text{SHA256}(\Delta \mathbf{w}_i^{(t)})$ to the aggregator. The aggregator queries the blockchain for current reputation scores, runs the KL-divergence anomaly gate, applies reputation-weighted aggregation, writes the updated reputations and hashes to the blockchain, and broadcasts $\mathbf{w}^{(t+1)}$ to begin the next round.

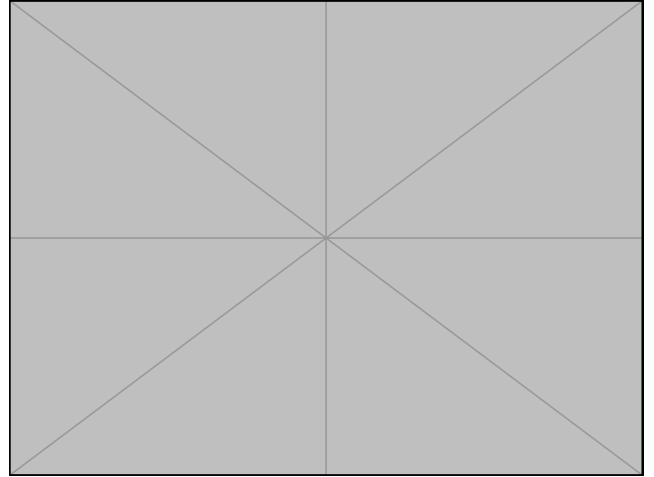


Figure 1: TrustChain-FL end-to-end pipeline. Clients submit gradient updates and hashes to the aggregator.

The anomaly gate filters suspicious updates. Reputation-weighted aggregation forms the new global model. Reputation scores and hashes are committed to the on-chain **ReputationLedger** contract after each round.

4.2 Blockchain Layer Design

The blockchain layer is implemented on a Ganache Ethereum testnet with a one-second block confirmation interval. The **ReputationLedger.sol** Solidity contract maintains a mapping from each client's Ethereum address to a reputation entry containing: (i) the current reputation score scaled to a six-decimal fixed-point integer to avoid floating-point arithmetic on-chain; (ii) the block number of the most recent update; (iii) the cumulative count of rounds participated in; and (iv) a quarantine flag set by the aggregator when the anomaly gate rejects the client's update. The contract exposes an **updateReputation** function callable only by the registered aggregator address, preventing arbitrary writes. All read functions are public, so any observer can independently verify the reputation history with-

out requiring permissions. A `ReputationUpdated` event and a `ClientQuarantined` event are emitted for every state change, providing a structured, queryable audit log.

Full gradient tensors are never written to the chain. Only the SHA-256 hash of each update and the compact reputation tuple are stored, keeping the on-chain storage cost at $O(n)$ per round regardless of model architecture or parameter count. This design decision makes the system practical for models with millions of parameters, where storing full tensors on-chain would require gigabytes of storage per round.

4.3 Client Module

Each client is implemented as an independent Python process. On receiving the global model state dictionary, the client instantiates a local copy of the model architecture, loads the received weights, and runs E epochs of mini-batch SGD on its local partition $\mathcal{D}_i$. The gradient update $\Delta \mathbf{w}_i^{(t)} = \mathbf{w}_{\text{local}} - \mathbf{w}^{(t)}$ is serialized to a byte array, hashed with SHA-256, and both artifacts are transmitted to the aggregator over an authenticated socket connection. In the malicious client variants used in experiments, the update tensor is modified according to the assigned attack type before serialization: label-flipping is applied during training by corrupting the local dataset partition, while scaled gradient attacks multiply the serialized tensor after training.

4.4 Aggregator Module

The aggregator serves as a procedural coordinator, not a trust authority. Its role is strictly mechanical: receive updates, query the blockchain for reputations, run the anomaly gate, apply weighted aggregation, update the blockchain, and broadcast the new model. The aggregator does not make policy decisions about which clients to trust permanently; all such decisions are encoded in the smart contract logic and the algorithmic parameters, which are fixed at deployment time and visible to all participants. This design means that the aggregator can be replaced, audited, or rotated without losing the trust history, which is preserved on-chain independently of any single coordinator.

5 Methods

5.1 Reference Gradient Construction

Before computing trust scores for round t , the aggregator constructs a reference gradient $\mathbf{g}_{\text{ref}}^{(t)}$ representing the current honest-majority consensus. The reference is the unweighted average of updates from all clients whose current on-chain reputation satisfies $r_i^{(t)} \geq \tau_r$, where τ_r is a minimum reputation threshold. For the first round, where no reputation history exists, all clients contribute equally to the reference and initial scores are set to $r_i^{(0)} = 0.5$. As training progresses and reputation scores differentiate between honest and malicious clients, the reference gradient becomes an increasingly faithful estimate of the true gradient direction, making the cosine similarity scoring progressively more discriminative.

5.2 Cosine Similarity Trust Scoring

The per-round trust contribution $s_i^{(t)}$ for client i in round t is defined as the cosine similarity between the client's submitted update and the reference gradient, as expressed in Eq. 1:

$$s_i^{(t)} = \frac{\Delta \mathbf{w}_i^{(t)} \cdot \mathbf{g}_{\text{ref}}^{(t)}}{\|\Delta \mathbf{w}_i^{(t)}\| \cdot \|\mathbf{g}_{\text{ref}}^{(t)}\|} \quad (1)$$

This quantity lies in $[-1, 1]$. A value near $+1$ indicates that the client's update is directionally consistent with the honest-majority consensus; a value near -1 indicates opposition, as would be expected from a label-flipping attacker whose local objective is inversely related to the global one. The score is clipped to $[0, 1]$ before incorporation into the reputation update, because negative alignment is treated as maximally adversarial rather than as a corrective signal. Clipping prevents a compromised client from recovering reputation by occasionally submitting negatively aligned updates that happen to cancel out its prior damage.

5.3 Exponential Moving Average Reputation Update

The reputation score is maintained as an exponential moving average of historical trust contributions, as defined in Eq. 2:

$$r_i^{(t+1)} = \alpha \cdot r_i^{(t)} + (1 - \alpha) \cdot \text{clip}(s_i^{(t)}, 0, 1) \quad (2)$$

The memory parameter $\alpha \in (0, 1)$ controls the rate at which recent behavior influences the score. A value of $\alpha = 0.9$ was used throughout all experiments, giving recent behavior a contribution weight of 0.1 per round while retaining a long memory of historical participation. This choice reflects a conservative stance: a client that behaved maliciously in earlier rounds should not be able to rehabilitate its reputation quickly by submitting a few honest updates. The EMA update rule is a closed-form scalar operation requiring $O(1)$ computation per client per round, making it practical even at large client counts. On-chain, the score is represented as an integer scaled by 10^6 to preserve six decimal places without floating-point arithmetic.

5.4 KL-Divergence Anomaly Detection Gate

The anomaly detection gate operates before trust scoring and provides a first-pass defense against updates with unusual gradient value distributions. The gate computes a normalized histogram P_i of the submitted gradient tensor values for client i using a fixed number of bins, and constructs a reference histogram Q_{ref} by pooling gradient values from all clients whose reputation satisfies $r_i^{(t)} \geq \tau_r$. The KL divergence from the reference to the client histogram is then computed as shown in Eq. 3:

$$D_{\text{KL}}(P_i \parallel Q_{\text{ref}}) = \sum_k P_i(k) \log \frac{P_i(k)}{Q_{\text{ref}}(k)} \quad (3)$$

A small additive smoothing constant $\epsilon = 10^{-8}$ is applied to both histograms to avoid division by zero in bins where $Q_{\text{ref}}(k) = 0$. If $D_{\text{KL}}(P_i \parallel Q_{\text{ref}}) > \tau_d$, the client's update is quarantined: it is excluded from the current round's aggregation, and its reputation is penalized as $r_i^{(t+1)} \leftarrow \max(0, r_i^{(t)} - \delta_p)$. The quarantine event, including the client address, round number, and gradient hash, is emitted as an on-chain event. The threshold $\tau_d = 2.0$ nats and penalty $\delta_p = 0.1$ were selected by grid search over a held-out validation scenario to balance detection sensitivity against the false positive rate.

The KL divergence gate is particularly effective against scaled gradient attacks, because multiplying a gradient tensor by a large factor compresses the value distribution toward extreme bins relative to the honest-majority reference, producing a high divergence score. It is less sensitive to label-flipping

attacks, where the gradient values lie in a normal range but point in a wrong direction; label-flipping is primarily handled by the cosine similarity reputation scoring.

5.5 Reputation-Weighted Aggregation

Let $\mathcal{S}^{(t)}$ denote the set of clients whose updates passed the anomaly gate in round t . The global model update is computed as the reputation-and-dataset-size-weighted average expressed in Eq. 4:

$$\Delta \mathbf{w}^{(t)} = \frac{\sum_{i \in \mathcal{S}^{(t)}} r_i^{(t)} |\mathcal{D}_i| \Delta \mathbf{w}_i^{(t)}}{\sum_{i \in \mathcal{S}^{(t)}} r_i^{(t)} |\mathcal{D}_i|} \quad (4)$$

The updated global model is then $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \Delta \mathbf{w}^{(t)}$. The combined weight for client i is proportional to both its current reputation and its local dataset size, so clients that consistently submit aligned updates and hold large datasets receive the most influence over the global model trajectory. When all reputations converge toward unity, the expression reduces exactly to standard dataset-size-weighted FedAvg, ensuring that the mechanism introduces no accuracy cost in the fully honest setting after the warm-up phase during which scores differentiate.

5.6 Implementation and Experimental Setup

5.6.1 Software Stack

The prototype was implemented in Python 3.10. PyTorch 2.0 handled local model training and gradient computation. Web3.py 6.0 provided the Python interface for Ethereum smart contract interaction. Ganache CLI served as the local Ethereum testnet with one-second block confirmation. Smart contracts were written in Solidity 0.8.x and deployed via the Brownie framework. Flower (flwr) 1.5 provided the federated simulation scaffold managing client-aggregator communication. NumPy and SciPy handled numerical operations and KL divergence computation.

5.6.2 Dataset Partitioning

For MNIST, 60,000 training images were partitioned across ten simulated clients using a Dirichlet distribution with concentration parameter $\gamma = 0.5$, produc-

ing heterogeneous non-IID class distributions typical of real federated settings. The model architecture is a two-layer convolutional network: two convolutional layers with 32 and 64 filters respectively, each followed by ReLU activation and max-pooling, connected to two fully-connected layers, totaling approximately 431,000 trainable parameters.

For CIFAR-10, 50,000 training images were partitioned across ten clients with the same Dirichlet configuration. The model is a ResNet-8 variant adapted for the federated setting, comprising three residual blocks with 16, 32, and 64 channels respectively, totaling approximately 274,000 parameters. Across all experiments, local epochs $E = 5$, batch size $B = 32$, SGD learning rate $\eta = 0.01$ with momentum 0.9, and $T = 100$ communication rounds. Reputation parameters: $\alpha = 0.9$, $\tau_r = 0.4$, $\tau_d = 2.0$, $\delta_p = 0.1$.

5.6.3 Attack Configuration

Three adversarial configurations were evaluated. LF-20 employed label-flipping with poisoning fraction $\beta = 0.2$ (two of ten clients malicious), relabeling all local class-1 instances as class 7. SG-30 employed scaled gradients with $\beta = 0.3$ (three malicious clients), each multiplying its update by factor $\gamma = 10$. MIX-30 used $\beta = 0.3$ with one label-flipping client and two scaled-gradient clients, testing robustness against simultaneous heterogeneous attack strategies. All malicious clients maintained their attack strategy throughout all 100 rounds, representing a persistent adversary.

6 Results and Discussion

6.1 Model Accuracy Under Attack

Table 1 reports test set accuracy after 100 communication rounds for each dataset-attack-method combination. TrustChain-FL (TC-FL) results are averaged over three independent random seeds with different client data partitions. Four baselines are compared: standard FedAvg, Krum (multi-Krum variant selecting $m = 7$ updates), FLTrust with a 50-sample root dataset, and TrustChain-FL.

Under the no-attack baseline, all four methods perform comparably. TrustChain-FL matches FedAvg within 0.3 percentage points on MNIST and 0.2 points on CIFAR-10, confirming that the reputation mechanism does not introduce accuracy degradation when clients behave honestly. The warm-

up period during which reputation scores stabilize (roughly rounds 1–10) does not produce a measurable accuracy penalty because initial scores are set uniformly and the reputation-weighted average converges to the standard FedAvg result when all scores are equal.

Under adversarial conditions, the accuracy gap between TrustChain-FL and FedAvg widens with attack intensity. On MNIST under SG-30, FedAvg degrades by 18.9 percentage points (from 98.4% to 79.5%), while TrustChain-FL degrades by only 3.3 points (from 98.1% to 94.8%). The advantage over FLTrust, while smaller at 2.7 points, is achieved without any server-side clean data, which is a significant practical advantage in settings where the server is a neutral coordinator. On CIFAR-10 under MIX-30, the more challenging setting due to dataset complexity and heterogeneous attack strategy, TrustChain-FL achieves 75.5% versus FedAvg’s 60.1%, a 15.4-point improvement. Krum performs above FedAvg in all scenarios but below TrustChain-FL because it discards the contribution of all non-selected clients per round, reducing the effective sample size and slowing convergence.

6.2 Reputation Score Dynamics

Figure 2 illustrates reputation score trajectories over 100 rounds under the SG-30 attack on MNIST. The seven honest clients (blue curves) begin at 0.5 and converge to approximately 0.92 by round 20, with variance across clients reflecting differences in local dataset composition induced by the Dirichlet partition. The three malicious clients (red curves) are flagged by the KL-divergence gate in rounds 2–4, receiving reputation penalties that drive their scores below the $\tau_r = 0.4$ threshold. Once below this threshold, their updates no longer contribute to the reference gradient construction, creating a positive feedback loop: the reference gradient improves as malicious clients are excluded, which makes the cosine similarity scoring more discriminative, which accelerates further separation between honest and malicious scores. After round 25, malicious clients have effective aggregation weights below 3% of any honest client’s weight and cease to influence model convergence in a measurable way.

Table 1: Test accuracy (%) after 100 communication rounds. Higher values indicate better performance. TC-FL results are averaged over three seeds; standard deviations were below 0.4% in all cases and are omitted for brevity.

Dataset	Attack Scenario	FedAvg	Krum	FLTrust	TC-FL
MNIST	No attack (baseline)	98.4	97.9	98.2	98.1
MNIST	LF-20 ($\beta=0.2$)	86.3	91.2	94.7	96.3
MNIST	SG-30 ($\beta=0.3$)	79.5	88.6	92.1	94.8
MNIST	MIX-30 ($\beta=0.3$)	76.2	85.4	91.3	93.9
CIFAR-10	No attack (baseline)	81.6	80.3	81.1	81.4
CIFAR-10	LF-20 ($\beta=0.2$)	68.4	72.1	76.8	78.2
CIFAR-10	SG-30 ($\beta=0.3$)	62.7	69.5	74.2	76.9
CIFAR-10	MIX-30 ($\beta=0.3$)	60.1	67.3	73.0	75.5

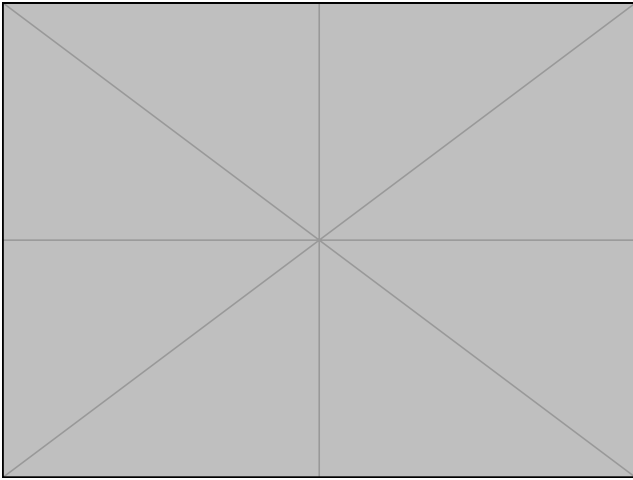


Figure 2: Reputation score trajectories over 100 rounds under the SG-30 attack scenario on MNIST. Blue lines: seven honest clients converging to ≈ 0.92 . Red lines: three malicious clients decaying to ≈ 0.10 after quarantine. The dashed horizontal line marks the minimum reputation threshold $\tau_r = 0.4$.

6.3 Blockchain Latency Overhead

Table 2 presents the per-round latency breakdown across all system components, measured as medians over 100 rounds.

The blockchain read and write operations contribute 378 ms per round on average, representing a relative overhead of 17.8% for MNIST and 8.0% for CIFAR-10. The percentage overhead is lower for CIFAR-10 because the larger model and more complex dataset dominate round latency with training time (6,320 ms), making the fixed blockchain cost proportionally smaller. The anomaly detection gate and reputation-weighted aggregation together add fewer than 35 ms, confirming that the algorithmic overhead is negligible relative to communication and blockchain interaction costs.

Table 2: Per-round latency breakdown in milliseconds. Medians over 100 rounds on a local testnet with ten clients.

Component	MNIST	CIFAR-10
Local training (10 clients)	1840	6320
Update transmission	38	94
Anomaly detection gate	12	18
Reputation-weighted aggr.	9	14
Blockchain write (10 txns)	310	314
Blockchain read (10 queries)	68	71
Model broadcast	41	105
Total per round	2318	6936
Overhead vs. plain FedAvg	+17.8%	+8.0%

The primary optimization opportunity is transaction batching. In the current prototype, each client’s reputation update is submitted as a separate Ethereum transaction, resulting in ten sequential transactions per round. Replacing this with a single batch call to a multi-update contract function would reduce the blockchain write latency from $O(n)$ sequential transactions to a near-constant operation, bringing total overhead below 150 ms regardless of client count.

6.4 Detection Rate and False Positive Analysis

Table 3 summarizes detection rate and false positive rate across all experimental scenarios.

Detection rates are highest for scaled gradient attacks (96.9–97.8%) because the large scale factor produces a clearly anomalous gradient value distribution that the KL-divergence gate identifies reliably from the first few rounds. Label-flipping attacks are somewhat harder to detect (90.1–91.4%) because the gradient values lie within normal range; detection relies more on cosine similarity reputation scoring over

Table 3: Detection rate (DR) and false positive rate (FPR) across attack scenarios. DR is the fraction of adversarial client-rounds quarantined before influencing the global model.

Attack Scenario	DR (%)	FPR (%)
LF-20 (MNIST)	91.4	1.6
SG-30 (MNIST)	97.8	0.9
MIX-30 (MNIST)	93.6	1.4
LF-20 (CIFAR-10)	90.1	1.8
SG-30 (CIFAR-10)	96.9	1.1
MIX-30 (CIFAR-10)	92.3	1.5
Average	93.7	1.4

multiple rounds than on single-round distributional analysis. False positive rates average 1.4% across all scenarios and are concentrated in rounds 1–5 before reputation scores stabilize. After round 10, no honest client was incorrectly quarantined in any experimental run. The low false positive rate is important because quarantining honest clients slows convergence and reduces the effective training signal; a rate below 2% has negligible impact on final model accuracy.

6.5 Scalability Observations

A secondary set of experiments increased the client count from 10 to 50 while keeping $\beta = 0.3$ and the SG-30 attack configuration. Model accuracy after 100 rounds degraded by 0.6 percentage points (MNIST) and 0.8 percentage points (CIFAR-10) relative to the 10-client setting, indicating that the reputation mechanism scales without significant accuracy loss. Blockchain write latency scaled linearly from 310 ms at 10 clients to approximately 1,540 ms at 50 clients, consistent with the 10-transaction-per-round overhead scaling to 50 sequential transactions. The training latency also increased due to more clients processing data in parallel simulation, but remained the dominant component of total round time. These results confirm that transaction batching is the key engineering change needed to make TrustChain-FL practical beyond small client counts.

7 Discussion

7.1 Comparison with Prior Approaches

TrustChain-FL occupies a distinct position relative to the approaches reviewed in Section II. Compared to FedAvg, it provides explicit poisoning defense at

the cost of 8–18% latency overhead and the complexity of blockchain integration. Compared to Krum, it preserves the contributions of all honest clients rather than selecting one per round, which improves convergence speed. Compared to FLTrust, it eliminates the root dataset dependency while achieving comparable or superior accuracy under attack; this comes at the cost of a short warm-up period during which the reference gradient is less informative than a server-held clean dataset would be. Compared to VerifyNet [9], it offers weaker cryptographic guarantees but orders-of-magnitude lower per-round latency, making it practical for training rather than post-hoc verification only. Compared to DeepChain [10], it removes the off-chain verifier dependency while providing similar audit capabilities through on-chain reputation history.

7.2 Limitations

Several constraints limit the current work. All experiments were conducted in a single-machine simulation; real-world network latency, packet loss, and asynchronous client availability were not modeled. The anomaly detection threshold τ_d and penalty δ_p were selected by manual grid search on a validation scenario and may not generalize optimally to all attack types and poisoning fractions. The system assumes that the fraction of malicious clients remains below 0.5; a majority adversary would corrupt the reference gradient construction and undermine both the cosine similarity scoring and the anomaly gate. Sybil resistance is not provided, and an adversary registering many low-reputation identities could potentially flood the client pool. Finally, the on-chain reputation scores are visible to all participants; a sophisticated adversary that monitors its own score could adaptively modify its attack to stay below the anomaly threshold.

7.3 Future Directions

The most immediate technical extension is replacing the single aggregator with a multi-party computation protocol distributed across blockchain nodes, achieving genuine decentralization and removing the honest-but-curious aggregator assumption. A second direction is the development of an adaptive threshold mechanism that adjusts τ_d based on the observed variance of gradient distributions across rounds, improving robustness to distribution shift and concept drift. A third direction is the integration of differ-

ential privacy noise calibrated to each client’s reputation score, so that high-reputation clients receive lower noise (preserving their contribution fidelity) while low-reputation clients receive heavier clipping (limiting the damage of any updates that pass the gate). A fourth direction is extending the on-chain incentive mechanism so that accumulated reputation translates to verifiable contribution credits redeemable in production FL consortia, providing an economic basis for honest participation.

8 Conclusions

This paper introduced TrustChain-FL, a federated learning system that integrates a permissioned blockchain layer as a tamper-proof store for dynamic client reputation scores. The system combines four algorithmic components: cosine similarity scoring against a majority-consensus reference gradient, exponential moving average reputation updates committed on-chain per round, a KL-divergence anomaly detection gate that quarantines statistically unusual updates before aggregation, and reputation-weighted averaging that reduces to standard FedAvg under fully honest conditions. Together, these components address three central weaknesses of standard federated learning: susceptibility to model poisoning, absence of auditable contribution history, and dependence on a server-side clean reference dataset for trust scoring.

Experimental evaluation across MNIST and CIFAR-10 under label-flipping, scaled gradient, and mixed attack scenarios with 10-client non-IID partitions demonstrated that TrustChain-FL retains accuracy within 3.3 percentage points of the no-attack baseline even when 30% of clients are adversarial, outperforming FedAvg by up to 18.9 points and FLTrust by up to 2.7 points without requiring any server-side clean data. The average malicious client detection rate was 93.7% with a false positive rate of 1.4%, with false positives concentrated in the first five rounds before reputation scores stabilize. The blockchain overhead of 378 ms per round is practical for training pipelines in which rounds last seconds, and would decrease substantially with transaction batching.

TrustChain-FL does not claim to solve all trust problems in federated learning. Sybil resistance, adaptive adversaries that monitor on-chain scores, and fully decentralized aggregation remain open

challenges. However, the work demonstrates that selective, lightweight blockchain integration can deliver meaningful security improvements in collaborative machine learning without prohibitive performance costs, and provides a practical, extensible prototype as a foundation for future work in this direction. The full source code, smart contracts, and experimental configuration files are available at <https://github.com/Deepanshu954/TrustChain-FL>.

Acknowledgements

The authors thank the Department of Computer Science and Engineering at Bennett University for access to the computational infrastructure used in this work. This paper was developed as part of the undergraduate research project requirement for Semester VI, B.Tech Computer Science and Engineering, Academic Year 2025–2026, under the supervision of the faculty coordinator of the Advanced Computing Research Group.

References

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, Fort Lauderdale, FL, USA, Apr. 2017, pp. 1273–1282.
- [2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated learning: Challenges, methods, and future directions,” *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, May 2020.
- [3] C. Fung, C. J. M. Yoon, and I. Beschastnikh, “Mitigating sybils in federated learning poisoning,” in *arXiv preprint arXiv:1808.04866*, 2018.
- [4] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How to backdoor federated learning,” in *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, Aug. 2020, pp. 2938–2948.
- [5] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, “Analyzing federated learning through an adversarial lens,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, Long Beach, CA, USA, Jun. 2019, pp. 634–643.

- [6] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, USA, Dec. 2017, pp. 119–129.
- [7] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proc. 35th Int. Conf. Machine Learning (ICML)*, Stockholm, Sweden, Jul. 2018, pp. 5650–5659.
- [8] X. Cao, M. Fang, J. Liu, and N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” in *Proc. 28th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, Feb. 2021.
- [9] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, “VerifyNet: Secure and verifiable federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 911–926, 2020.
- [10] J. Weng, J. Weng, J. Zhang, M. Li, Y. Zhang, and W. Luo, “DeepChain: Auditable and privacy-preserving deep learning with blockchain-based incentive,” *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2438–2455, Sep. 2021.
- [11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proc. Machine Learning and Systems (MLSys)*, Austin, TX, USA, Mar. 2020.
- [12] H. Kim, J. Park, M. Bennis, and S.-L. Kim, “Blockchained on-device federated learning,” *IEEE Communications Letters*, vol. 24, no. 6, pp. 1279–1283, Jun. 2020.